

Introduction to RISC-V assembly language

(reference: text Sections 2.1-2.3)

RISC-V has a modular design, with a number of alternative “base” parts and optional “extensions”

for the **base**, we will be using **RV32I**

example **extensions** are the **F** and **D extensions** for floating point, but there are many others

see [here](#) for RISC-V instruction set manual

registers and **memory**

in assembly language, programmer works directly with hardware storage elements called **registers**

- high-speed storage locations on the processor chip ... only have a small number of these

registers used to store data that the program is currently operating on (also some special purpose uses)

assembly language programmer also uses a **memory space**

- implemented using the computer system's "main memory" (typically implemented on separate chips than the processor and much, much slower to access than the registers ... but have a lot of it! e.g. 8 – 32 GB on a typical laptop), plus (as we'll see later in course) processor cache memory

In the **RISC-V** base we will be using (RV32I), have **32 registers**, each **32 bits** (inside back cover of text has listing)

the **memory space** (in RV32I) can be viewed as an **array of 8-bit bytes**

- the index of a byte in this array is its **memory address**
- the first byte in memory has memory address 0, the second byte has memory address 1, the third byte has address 2, and so on ...
- memory addresses are 32 bits, the same as the register size ($\Rightarrow$ largest possible address is $2^{32}-1$)